

Homework 8

PROBLEM 1

The data sequence is

$$x[n] = 0 \quad \text{for } n < 0 \text{ or } n > 626.$$

Its discrete-time Fourier transform (DTFT) is

$$X(\omega) = \sum_{n=0}^{626} x[n] e^{-j\omega n}.$$

We want the samples

$$X(\omega_k), \quad \omega_k = \frac{2\pi}{627} + \frac{2\pi k}{256}, \quad k = 0, 1, \dots, 255,$$

but we can only compute a 256-point DFT. So we can try to write each desired DTFT sample in the form of a 256-point DFT of a different sequence $g[n]$.

Start by plugging ω_k into the DTFT:

$$X(\omega_k) = \sum_{n=0}^{626} x[n] e^{-j\left(\frac{2\pi}{627} + \frac{2\pi k}{256}\right)n} = \sum_{n=0}^{626} x[n] e^{-j\frac{2\pi}{627}n} e^{-j\frac{2\pi k}{256}n}.$$

The factor with k in it, $\exp\left(-j\frac{2\pi k}{256}n\right)$, is exactly what appears in a 256-point DFT, except that the DFT only sums n from 0 to 255. Our sequence $x[n]$ has length 627, so we split the sum into blocks of length 256.

Write

$$n = 256m + r, \quad m \in \{0, 1, 2\}, \quad r \in \{0, 1, \dots, 255\},$$

with the restriction $256m + r \leq 626$. Then

$$\begin{aligned} X(\omega_k) &= \sum_{n=0}^{626} x[n] e^{-j\frac{2\pi}{627}n} e^{-j\frac{2\pi k}{256}n} \\ &= \sum_{r=0}^{255} \sum_{\substack{m \in \{0,1,2\} \\ 256m+r \leq 626}} x[256m+r] e^{-j\frac{2\pi}{627}(256m+r)} e^{-j\frac{2\pi k}{256}(256m+r)}. \end{aligned}$$

Now use

$$e^{-j\frac{2\pi k}{256}(256m+r)} = e^{-j\frac{2\pi k}{256}r} e^{-j2\pi km} = e^{-j\frac{2\pi k}{256}r} \cdot 1,$$

since $e^{-j2\pi km} = 1$ for integers k, m . So

$$X(\omega_k) = \sum_{r=0}^{255} \left[\sum_{\substack{m \in \{0,1,2\} \\ 256m+r \leq 626}} x[256m+r] e^{-j\frac{2\pi}{627}(256m+r)} \right] e^{-j\frac{2\pi k}{256}r}.$$

This already has the structure of a 256-point DFT in the variable r . So define a new sequence $g[n]$ of length 256 by

$$g[r] = \sum_{\substack{m \in \{0,1,2\} \\ 256m+r \leq 626}} x[256m+r] e^{-j\frac{2\pi}{627}(256m+r)}, \quad r = 0, 1, \dots, 255.$$

Then

$$X(\omega_k) = \sum_{r=0}^{255} g[r] e^{-j\frac{2\pi k}{256}r},$$

which is exactly the 256-point DFT of $g[r]$:

$$G[k] = X(\omega_k), \quad k = 0, 1, \dots, 255.$$

It is sometimes helpful to write $g[r]$ explicitly in terms of $x[n]$. For each r there are up to three contributing terms:

$$g[r] = x[r] e^{-j \frac{2\pi}{627} r} + x[r + 256] e^{-j \frac{2\pi}{627} (r+256)} + x[r + 512] e^{-j \frac{2\pi}{627} (r+512)},$$

with the understanding that $x[n] = 0$ whenever $n > 626$. Equivalently:

$$\begin{cases} g[r] = x[r] e^{-j \frac{2\pi}{627} r} + x[r + 256] e^{-j \frac{2\pi}{627} (r+256)} + x[r + 512] e^{-j \frac{2\pi}{627} (r+512)}, & 0 \leq r \leq 114, \\ g[r] = x[r] e^{-j \frac{2\pi}{627} r} + x[r + 256] e^{-j \frac{2\pi}{627} (r+256)}, & 115 \leq r \leq 255. \end{cases}$$

So the procedure is:

1. Form the length-256 sequence $g[n]$ from $x[n]$ using the formula above (stacking $x[n]$ in blocks of 256 samples and multiplying each term by $e^{-j2\pi n/627}$).

2. Run the 256-point FFT on $g[n]$ to obtain $G[k]$.

The resulting DFT values satisfy

$$G[k] = X\left(\frac{2\pi}{627} + \frac{2\pi k}{256}\right), \quad k = 0, \dots, 255,$$

which is exactly the set of desired DTFT samples.

PROBLEM 2

The input length is

$$L_x = 8000,$$

and the impulse response length

$$M = 100.$$

The FFT size we are allowed to use in parts (a) and (b) is

$$N = 256.$$

In both overlap-add and overlap-save, we want each circular convolution of length N to be equivalent to a linear convolution of a block of the input with the length- M filter. For a given N and M the maximum number of new (non-overlapped) output samples per block is

$$L = N - M + 1 = 256 - 100 + 1 = 157.$$

So in each block we can produce at most $L = 157$ usable samples of the overall output without time aliasing.

The overall linear convolution has length

$$L_y = L_x + M - 1 = 8000 + 100 - 1 = 8099.$$

(a) Overlap-add. In overlap-add we split the input $x[n]$ into non-overlapping segments of length L (except possibly the last one), zero-pad each segment to length N , convolve with $h[n]$, and then add the shifted blocks in time.

Number of blocks:

$$B_{\text{OA}} = \left\lceil \frac{L_x}{L} \right\rceil = \left\lceil \frac{8000}{157} \right\rceil = 51.$$

For each block we need

- one 256-point DFT of the input block,
- one 256-point IDFT of the product $X_k(\omega)H(\omega)$.

The filter spectrum $H[k]$ can be computed once and reused.

So the minimum numbers of transforms are

$$\text{DFT's: } B_{\text{OA}} + 1 = 51 + 1 = 52, \quad \text{IDFT's: } B_{\text{OA}} = 51.$$

(b) Overlap-save. In overlap-save we process blocks of length N that overlap in time. Each block produces $L = N - M + 1 = 157$ valid output samples; the first $M - 1$ output samples of each circular convolution are discarded.

We can think of $x[n]$ as being padded with $M - 1 = 99$ leading zeros. The total length we need to cover is then $L_x + M - 1 = 8099$. Since each block advances by L new samples, the number of blocks is

$$B_{\text{OS}} = \left\lceil \frac{L_x + M - 1}{L} \right\rceil = \left\lceil \frac{8099}{157} \right\rceil = 52.$$

Again,

- each block needs one 256-point DFT and one 256-point IDFT,
- $H[k]$ is computed once and reused.

Therefore

$$\text{DFT's: } B_{\text{OS}} + 1 = 52 + 1 = 53, \quad \text{IDFT's: } B_{\text{OS}} = 52.$$

(c) Multiplication counts. For a power-of-two length N , an N -point DFT or IDFT requires

$$C_{\text{FFT}}(N) = \frac{N}{2} \log_2 N$$

complex multiplications.

We also need N complex multiplications per block to do the frequency-domain product $Y[k] = X[k]H[k]$.

Overlap-add, $N = 256$. For $N = 256$,

$$C_{\text{FFT}}(256) = \frac{256}{2} \log_2 256 = 128 \cdot 8 = 1024$$

multiplications per DFT or IDFT.

From part (a), overlap-add uses

$$N_{\text{DFT}} = 52, \quad N_{\text{IDFT}} = 51, \quad B_{\text{OA}} = 51 \text{ blocks.}$$

Total multiplications:

$$\begin{aligned} C_{\text{OA}} &= (N_{\text{DFT}} + N_{\text{IDFT}}) C_{\text{FFT}}(256) + B_{\text{OA}} \cdot N \\ &= (52 + 51) \cdot 1024 + 51 \cdot 256 \\ &= 103 \cdot 1024 + 51 \cdot 256 \\ &= 105,472 + 13,056 \\ &= 118,528 \text{ multiplications (complex).} \end{aligned}$$

Overlap-save, $N = 256$. From part (b),

$$N_{\text{DFT}} = 53, \quad N_{\text{IDFT}} = 52, \quad B_{\text{OS}} = 52.$$

Total:

$$\begin{aligned} C_{\text{OS}} &= (N_{\text{DFT}} + N_{\text{IDFT}}) C_{\text{FFT}}(256) + B_{\text{OS}} \cdot N \\ &= (53 + 52) \cdot 1024 + 52 \cdot 256 \\ &= 105 \cdot 1024 + 52 \cdot 256 \\ &= 107,520 + 13,312 \\ &= 120,832 \text{ multiplications.} \end{aligned}$$

So with $N = 256$ the overlap-add method is slightly more efficient than overlap-save because it needs one fewer block.

Direct convolution. Using straightforward time-domain convolution, there are $L_y = 8099$ output samples and each requires $M = 100$ multiplications, so

$$C_{\text{direct}} = L_y \cdot M = 8099 \cdot 100 = 809,900$$

multiplications.

Using one $N = 8192$ point FFT. Now suppose we use 8192-point DFTs/IDFTs instead, so that the entire convolution can be done in one shot (no blocking). We zero-pad $x[n]$ and $h[n]$ to length $N = 8192$, compute their DFTs, multiply, then apply one IDFT.

For $N = 8192$,

$$C_{\text{FFT}}(8192) = \frac{8192}{2} \log_2 8192 = 4096 \cdot 13 = 53,248.$$

We need

$$N_{\text{DFT}} = 2 \quad (\text{for } x[n] \text{ and } h[n]), \quad N_{\text{IDFT}} = 1,$$

plus $N = 8192$ pointwise multiplications.

Total:

$$\begin{aligned} C_{8192} &= (2 + 1) C_{\text{FFT}}(8192) + 8192 \\ &= 3 \cdot 53,248 + 8192 \\ &= 159,744 + 8192 \\ &= 167,936 \text{ multiplications.} \end{aligned}$$

So both 256-point FFT-based streaming methods are more efficient than direct convolution. Among the FFT-based approaches, overlap-add with $N = 256$ is the most efficient here; using a single 8192-point FFT costs more multiplications than the 256-point block methods, even though it actually avoids block-management.

PROBLEM 3 (3.26 IN THE BOOK)

We are told that $x[n]$ is right-sided and that its z -transform is

$$X(z) = \frac{1}{(1 - az^{-1})(1 - bz^{-1})} = \frac{z^2}{(z - a)(z - b)}.$$

Because $x[n]$ is right-sided and the poles are at $z = a$ and $z = b$, the ROC must be

$$|z| > \max\{|a|, |b|\}.$$

The usual z -transform pair for a right-sided exponential is

$$a^n u[n] \longleftrightarrow \frac{1}{1 - az^{-1}} = \frac{z}{z - a}, \quad |z| > |a|.$$

So it is convenient to write $X(z)$ as a linear combination of the terms $z/(z - a)$ and $z/(z - b)$.

Partial fraction expansion in z . Assume

$$X(z) = \frac{z^2}{(z - a)(z - b)} = A \frac{z}{z - a} + B \frac{z}{z - b}.$$

Bring to a common denominator:

$$A \frac{z}{z - a} + B \frac{z}{z - b} = \frac{Az(z - b) + Bz(z - a)}{(z - a)(z - b)}.$$

Therefore the numerators must match:

$$z^2 = Az(z - b) + Bz(z - a) = z[(A + B)z - (Ab + Ba)].$$

Cancel a factor of z and equate coefficients of like powers of z :

$$\begin{cases} A + B = 1, \\ Ab + Ba = 0. \end{cases}$$

From $Ab + Ba = 0$ we get $B = -\frac{b}{a}A$ (assuming $a \neq 0$). Insert this into $A + B = 1$:

$$A - \frac{b}{a}A = 1 \quad \Rightarrow \quad A \left(1 - \frac{b}{a}\right) = 1 \quad \Rightarrow \quad A = \frac{a}{a - b}.$$

Then

$$B = -\frac{b}{a}A = -\frac{b}{a} \cdot \frac{a}{a - b} = -\frac{b}{a - b} = \frac{b}{b - a}.$$

So the partial fraction expansion in terms of z is

$$X(z) = \frac{a}{a - b} \frac{z}{z - a} + \frac{b}{b - a} \frac{z}{z - b}.$$

Inverse z -transform. Using the basic pair

$$\frac{z}{z - a} \longleftrightarrow a^n u[n], \quad \frac{z}{z - b} \longleftrightarrow b^n u[n],$$

we immediately get

$$x[n] = \frac{a}{a - b} a^n u[n] + \frac{b}{b - a} b^n u[n].$$

It is nice to clean this up a bit. Since $\frac{b}{b - a} = -\frac{b}{a - b}$, we can write

$$x[n] = \frac{a^{n+1}}{a - b} u[n] - \frac{b^{n+1}}{a - b} u[n] = \frac{a^{n+1} - b^{n+1}}{a - b} u[n].$$

So the right-sided sequence whose z-transform is $X(z) = 1/[(1 - az^{-1})(1 - bz^{-1})]$ is

$$x[n] = \frac{a^{n+1} - b^{n+1}}{a - b} u[n], \quad |z| > \max\{|a|, |b|\}.$$

PROBLEM 4: LOWPASS IIR IN MATLAB

We need to design four minimum-order IIR lowpass filters that satisfy

$$\begin{aligned}\omega_p &= 0.4\pi, & \omega_s &= 0.5\pi, \\ \delta_1 &= 0.1, & \delta_2 &= 0.01.\end{aligned}$$

And we know

$$f_p = \frac{\omega_p}{\pi} = 0.4, \quad f_s = \frac{\omega_s}{\pi} = 0.5.$$

The passband ripple and stopband attenuations are expressed as

$$R_p = -20 \log_{10}(1 - \delta_1), \quad R_s = -20 \log_{10}(\delta_2).$$

Plugging in the tolerances gives

$$R_p \approx 0.915 \text{ dB}, \quad R_s = 40 \text{ dB}.$$

Using `designfilt`, we can design the following:

1. Equiripple (FIR, linear phase)
2. Butterworth
3. Chebyshev II
4. elliptic

Here is the matlab code...

```
% Filter specs
fp = 0.4;          % normalized passband edge
fs = 0.5;          % normalized stopband edge
Rp = -20*log10(1-0.1); % passband ripple in dB
Rs = -20*log10(0.01); % stopband attenuation in dB

% Equiripple FIR (minimum order)
d_equ = designfilt('lowpassfir', ...
    'PassbandFrequency', fp, ...
    'StopbandFrequency', fs, ...
    'PassbandRipple', Rp, ...
    'StopbandAttenuation', Rs, ...
    'DesignMethod','equiripple');

% Butterworth (minimum order)
d_butt = designfilt('lowpassiir', ...
    'FilterOrder','minimum', ...
    'PassbandFrequency', fp, ...
    'StopbandFrequency', fs, ...
    'PassbandRipple', Rp, ...
    'StopbandAttenuation', Rs, ...
    'DesignMethod','butter');

% Chebyshev II (minimum order)
d_cheb2 = designfilt('lowpassiir', ...
    'FilterOrder','minimum', ...
    'PassbandFrequency', fp, ...
    'StopbandFrequency', fs, ...
    'PassbandRipple', Rp, ...
    'StopbandAttenuation', Rs, ...
    'DesignMethod','cheby2');

% Elliptic
d_ellip = designfilt('lowpassiir', ...
    'FilterOrder','minimum', ...
```

```

'PassbandFrequency', fp, ...
'StopbandFrequency', fs, ...
'PassbandRipple', Rp, ...
'StopbandAttenuation', Rs, ...
'DesignMethod','ellip');

% Determine filter orders
order_equ = fildord(d_equ)
order_butt = fildord(d_butt)
order_cheb2 = fildord(d_cheb2)
order_ellip = fildord(d_ellip)

% Plot responses
[H1, w] = freqz(d_equ, 2048);
H2 = freqz(d_butt, 2048);
H3 = freqz(d_cheb2, 2048);
H4 = freqz(d_ellip, 2048);

figure; hold on; grid on;
plot(w/pi, 20*log10(abs(H1)));
plot(w/pi, 20*log10(abs(H2)));
plot(w/pi, 20*log10(abs(H3)));
plot(w/pi, 20*log10(abs(H4)));

ylim([-100 2]);
xlabel('Normalized Frequency (\omega/\pi)');
ylabel('Magnitude (dB)');
legend('Equiripple FIR','Butterworth','Chebyshev II','Elliptic');
title('Magnitude Responses of Minimum-Order Filters');

```

Filter Orders.

$$\text{order}_{\text{equiripple}} = 31$$

$$\text{order}_{\text{butter}} = 17$$

$$\text{order}_{\text{cheb2}} = 8$$

$$\text{order}_{\text{ellip}} = 5$$

